MODULE IV · WEEK 11 · LLMs IN THE RESEARCH PIPELINE

LLM annotation and distillation

From bag-of-words to a small model you can freeze and re-run

M. Fuat Kına

TODAY

Four steps, one pipeline

01	**Text as data**	How we have turned words into numbers, and what each step gained and lost.
02	**The architecture**	Attention, transformers, and the three model families you will meet.
03	**The LLM as annotator**	Prompting as a codebook, APIs in practice, and validating against human coding.
04	**Distillation**	Using the expensive model to train a small one you can run yourself.

The thread running through all four: an LLM is useful to us as a measurement instrument, and instruments have to be checked.

TEXT AS DATA

Layers of representation

Each layer captures more meaning — and is harder to look inside.

APPROACH	IDEA	GAIN	COST
Dictionaries	Counting words from a predefined list	Transparent	No context
Bag-of-words, TF-IDF	Frequency vectors	Strong baseline	No word order
Topic models (LDA)	Latent topic distributions	Good for discovery	Fragile interpretation
Static embeddings	word2vec, GloVe	Semantic similarity	One vector per word
Contextual embeddings	BERT and relatives	Handles polysemy	Opaque, costly
Generative LLMs	Task defined by instruction	Works with few examples	Needs validation

HOW WE GOT HERE

From counting words to predicting them

1990s

n-grams, RNNs

Sequential models. Long sentences are a problem; training cannot be parallelised.

2017

Transformers

"Attention is all you need." Sequential computation disappears, so scaling becomes possible.

2018–19

BERT and friends

Pre-train on a large corpus, then fine-tune on a small labelled set for your own task.

2020s

Generative LLMs

Trained on web-scale text. The task can now be described in words rather than in labels.

What an LLM does is still one thing: predict the next token. Everything else follows from doing that at scale.

THE ARCHITECTURE

Attention, in plain terms

The idea

- Every token can look at every other token in the text.
- The model learns how much weight to give each one.
- "Bank" gets a different representation next to "river" than next to "loan".
- Nothing is processed in sequence, so the whole thing parallelises — which is why it scaled.

In five steps

1	**Tokenize**	break text into subwords
2	**Embed**	turn each piece into numbers
3	**Attend**	each token looks at the others
4	**Predict**	score every possible next token
5	**Repeat**	pick one, append it, start again

THREE FAMILIES

Which model for which job?

The distinction matters more for us than any benchmark score.

ENCODER-ONLY

BERT, RoBERTa, XLM-R

Reads the whole text in both directions and produces a representation. Natural fit for classification and measurement. Small, runs locally, can be frozen.

Our student model

DECODER-ONLY

GPT, Llama, Mistral, Qwen

Generates text and follows instructions. Natural fit for annotation with a written codebook, and for simulation.

Our teacher model

ENCODER–DECODER

T5, BART

Maps one sequence to another. Summarisation, translation, structured rewriting.

Occasional use

The week ends by connecting the second to the first: the decoder teaches, the encoder is what you keep.

VOLUME

From BERT to LLMs

BERT is small but trainable, LLMs are powerful but not easily trainable

110M

BERT-base / BERTurk

12 layers, 768 dims, ~440 MB

340M

BERT-large

24 layers, 1024 dims

7–8B

A small open LLM

~70× BERT-base

~2 GB

Full fine-tune of BERT-base

fits a free Colab GPU (8B LLM: ~120 GB)

What is a parameter?

- A number the model learns from data, like β in a regression.
- BERT-base: ~110M of them, in weight matrices.
- Hyperparameters (learning rate, epochs, batch) are set by you, not learned.

Why it gives probabilities

- The classification head on top of BERT is a logistic regression on the [CLS] vector.
- Trained with cross-entropy; 1/0 comes from a threshold.
- Scores are often overconfident: check calibration; tune the threshold on validation data.

Learning rate and data size

- Safe range 1e-5 to 5e-5. Too high: forgets its pre-training; too low: learns nothing.
- 1,000 labels: ~190 steps; try more epochs, warmup, several seeds.
- 20,000 labels: ~3,750 steps; 2e-5 and 2–3 epochs usually suffice.

Fine-tuning BERT = learned features + a logistic regression, trained together. Tune on validation data; report on the gold set.

FOUR WAYS TO USE ONE

Prompt, or train?

ROUTE	WHAT IT IS	WHY	WATCH OUT
Zero-shot	Just ask	No labelled data, instant	Unstable wording, weak on niche tasks
Few-shot	Show 2–5 examples first	Large gain for little cost	Eats context; examples bias output
Fine-tuning	Change the weights	Best on a narrow, repeated task	Needs hundreds of labels and GPU time
RAG	Retrieve your documents first	Adds facts the model never saw	Capped by retrieval; new facts, not new skills

A sensible order: start zero-shot, add examples, and only fine-tune once you know the task is stable and repeated.

THE LLM AS ANNOTATOR

The prompt is a codebook

What used to be written for student coders is now written for a model.

What goes in it

- The concept, defined — not just named
- Inclusion and exclusion criteria
- Borderline cases, decided in advance
- The label set, and an option for "unclear"
- The output format — ask for JSON

Why treat it that way

If the prompt is the instrument, then the prompt version is the instrument version. It belongs in the appendix of the paper, in full.

Rewording changes the labels, so a small prompt-sensitivity check is a robustness check, not a nicety.

IN PRACTICE

The knobs that matter

Temperature

0 for labelling, higher for generation. Keep it low and state it.

Model version

Pin it. The model you ran in April is not the one you run in October.

Structured output

Ask for JSON with a fixed label set. Do not parse free text.

Cost and batching

You pay per token both ways. Batch, and keep prompts tight.

Repeated calls

Label the same item k times and take the majority — a cheap reliability check.

Open vs. closed

A local open-weight model costs more setup and buys reproducibility.

WHAT GOES WRONG

Known failure modes

None of these are exotic. Expect all of them.

Hallucination

Confident, fluent, wrong — and most likely exactly where the domain is obscure.

Central-tendency bias

Ask for Likert answers and you get a pile of 3s and 4s. Real distributions have tails.

Language gap

Performance drops outside English. Test on Turkish before you trust the numbers.

Prompt sensitivity

Rewording the instruction changes the labels. Document versions, run ablations.

Essentialization

Stereotypes baked into training data resurface around gender, region, religion.

Version drift

Silent updates break reproducibility. Store outputs, record the date.

VALIDATION

No gold standard, no measurement

A few hundred items, coded carefully by humans. That is the ruler for everything that follows.

Build it properly

Two independent coders, a reconciliation rule, and a codebook you revise as you go.

Report the right metrics

Precision, recall and F1 per class; Cohen's κ or Krippendorff's α for agreement.

Read the confusion matrix

Overall accuracy hides collapse in the small classes — which are often the interesting ones.

Compare models side by side

Same items, several models and prompts. Differences in error structure matter more than F1.

Gilardi et al. (2023) report that a model beat crowd workers on their tasks. Whether it beats your coders on your task is an empirical question about your data.

USAGE GUIDELINES

Guidelines for LLM annotation

Two guidelines, one message: treat the model like a new coder you have to train, test and document.

Keep humans in the loop

The LLM is one coder among several. Combine its labels with a small gold set (e.g. DSL) rather than trusting raw LLM counts.

Iterate prompt and codebook together

Code a sample with humans and the LLM, study disagreements, revise both. Archive it all as a “prompt codebook”.

Prompt for structure, report everything

Context → question → constraints; lettered options; JSON output; temperature ≈ 0. Report every parameter.

Choose the model deliberately

Prefer open models you can pin and run yourself; never the web chat. Check that results hold across models.

Validate on data the prompt never saw

Freeze the prompt first. 20–30+ items per class; per-class F1 and κ, benchmarked against human agreement.

Know where it breaks

Weak on expert taxonomies and long texts. Test prompt paraphrases (Krippendorff’s α); skip public labelled sets it may have memorised.

Ziems, C., Held, W., Shaikh, O., Chen, J., Zhang, Z., & Yang, D. (2024). Can large language models transform computational social science? Computational Linguistics, 50(1), 237–291. · Törnberg, P. (2024). Best practices for text annotation with large language models. Sociologica, 18(2), 145–159.

THE SCALE PROBLEM

Three million texts, and a budget

LLM FOR EVERYTHING

- Cents per item add up: 3M items is a five-figure bill
- Rate limits, outages, changing terms of service
- Version drift between April and October
- Sensitive data leaves your institution

A SMALL LOCAL MODEL

- Runs on one GPU; free at inference time
- Your own weights — freeze them and re-run in five years
- Data stays where your ethics approval says it stays
- But it needs labelled data. That is the catch.

So: one model is too expensive to run everywhere, the other is too data-hungry to train. The next slide is the way out.

THE HYBRID

Let the LLM teach a BERT

Use the expensive model once, to make the cheap model good.

1

Hand-label ~100 items

Careful human coding. This is the gold set and it never moves into training.

2

Validate the LLM on them

Choose the model and prompt whose labels match yours. If none do, fix the codebook first.

3

Run it on a few thousand

These are silver labels: noisier, but far more numerous, and affordable.

4

Fine-tune and freeze

Train the encoder on the silver set, test it against the gold set, then label everything.

One point worth repeating: the student is tested against the human gold set, never against the teacher. Matching the teacher only proves it copied well.

BEFORE YOU REPORT

A short checklist — and when to skip all this

Put in the paper

- Model name, version, access date
- The full prompt, and any variants tried
- Temperature, seed, number of repeats
- Gold-set size and coder agreement
- Per-class metrics, not just accuracy
- Code and labels in the repository

When distillation is the wrong tool

- The task genuinely needs reasoning, not classification
- The category is very rare in the corpus
- The concept shifts over the period you study
- You only have a few thousand documents anyway — then just run the LLM

WEEK 11 · CLOSING

A language model is a measurement instrument. Instruments get validated.

READING

- Gilardi, Alizadeh & Kubli (2023), PNAS 120(30)
- Ziems et al. (2024), Computational Linguistics 50(1)
- Jurafsky & Martin, chapters on transformers and LLMs
- Optional: Törnberg (2024), arXiv:2402.05129

ASSIGNMENT 5 — RELEASED

Annotate a small corpus with a language model, validate it against a human-coded gold standard, and distil the labels into a supervised classifier.

Next week: synthetic agents — can a model stand in for a respondent?

mkina@ku.edu.tr